

Containerization & DevOps Lab

Document: Lab12.md | Containerization & DevOps Coursework

Experiment 12 — Study and Analyse Container Orchestration using Kubernetes

Objective

To understand why Kubernetes is used for container orchestration and learn how to deploy, scale, and manage applications using Kubernetes commands.

Theory

Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry Standard	Most companies use Kubernetes
Powerful Scheduling	Automatically decides where to run applications
Large Ecosystem	Many tools and plugins available
Cloud Native Support	Works on AWS, Google Cloud, Azure, etc.

Kubernetes has become the **dominant orchestration platform for modern cloud applications**.

Core Kubernetes Concepts

Docker Concept	Kubernetes Equivalent	Meaning
Container	Pod	Smallest deployable unit containing one or more containers
Compose Service	Deployment	Describes how applications run and how many replicas
Load Balancer	Service	Exposes application to external traffic
Scaling	ReplicaSet	Ensures a desired number of pods run

A **Pod** is the smallest execution unit in Kubernetes.

A **Deployment** manages pod creation and scaling.

A **Service** exposes pods to users or other services.

Hands-On Lab

This lab assumes:

- `kubectl` installed
- Kubernetes cluster running (Minikube or k3d)

Task 1 — Create a Deployment

A Deployment specifies:

- container image
- number of replicas
- labels used to identify pods

Create file:

```

yuvraj_singh@LUCIF3R: ~/
GNU nano 7.2                               wordpress-deployment.yaml *
# wordpress-deployment.yaml
apiVersion: apps/v1                        # Which Kubernetes API to use
kind: Deployment                           # Type of resource
metadata:
  name: wordpress                         # Name of this deployment
spec:
  replicas: 2                             # Run 2 identical pods
  selector:
    matchLabels:
      app: wordpress                     # Pods with this label belong to this deployment
  template:
    metadata:
      labels:
        app: wordpress                 # Label applied to each pod
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest         # Docker image
        ports:
        - containerPort: 80             # Port inside the container

```

Apply deployment:

```
yuvraj_singh@LUCIF3R: /Lab12s kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress unchanged
```

Kubernetes will create **two pods running WordPress**.

Task 2 — Expose the Deployment

Pods are temporary and may change.

A **Service** provides a stable network endpoint.

Create file:

```

yuvraj_singh@LUCIF3R: ~/
GNU nano 7.2                               wordpress-service.yaml *
# wordpress-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort                          # Exposes service on a port of each node (VM)
  selector:
    app: wordpress                       # Send traffic to pods with this label
  ports:
    - port: 80                           # Service port
      targetPort: 80                     # Pod port
      nodePort: 30007                   # External port (range: 30000-32767)

```

Apply service:

```
yuvraj_singh@LUCIF3R: ab12; kubectl apply -f wordpress-service.yaml
service/wordpress-service created
```

The application is now accessible via a node port.

Task 3 — Verify Deployment

Check pods:

```
yuvraj_chaula@LUCIF3R:~/Lab12; kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-2vjp8	1/1	Running	0	5m43s
wordpress-6698dd7d66-5x6k6	1/1	Running	0	5m43s

Expected output:

```
NAME READY STATUS RESTARTS AGE
wordpress-xxxxx-yyyyy 1/1 Running 0 1m
wordpress-xxxxx-zzzzz 1/1 Running 0 1m
```

Check service:

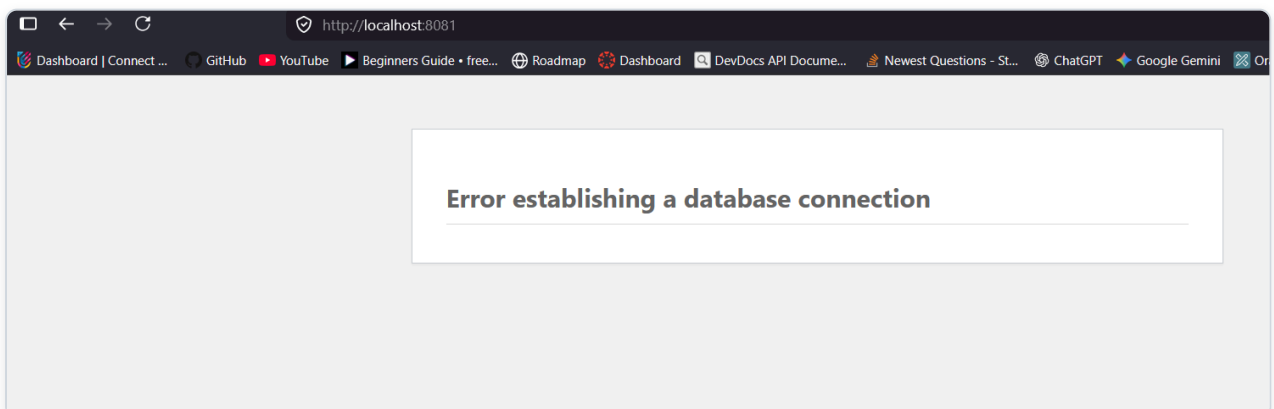
```
yuvraj_singh@LUCIF3R:~/Lab12$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	6m40s
wordpress-service	NodePort	10.102.220.78	<none>	80:30007/TCP	86s

Example output:

```
NAME TYPE CLUSTER-IP PORT(S)
wordpress-service NodePort 10.43.x.x 80:30007/TCP
```

Access application:



Node IP can be obtained using:

```
minikube ip
```

or using `localhost` for k3d clusters.

Task 4 — Scale the Deployment

Increase replicas from **2 to 4**:

```
anirudh_chawla@LUCIF3R:~/Lab12$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
```

Verify:

```
yuvraj_singh@LUCIF3R: /Lab12; kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-2vjp8	1/1	Running	0	10m
wordpress-6698dd7d66-5x6k6	1/1	Running	0	10m
wordpress-6698dd7d66-8p55z	1/1	Running	0	27s
wordpress-6698dd7d66-q927g	1/1	Running	0	27s

Four pods should now be running.

Scaling allows the application to handle **higher traffic and improved performance**.

Task 5 — Self-Healing Demonstration

Kubernetes automatically replaces failed pods.

```
yuvraj_chaula@LUCIF3R:~/Lab12; kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-2vjp8	1/1	Running	0	10m
wordpress-6698dd7d66-5x6k6	1/1	Running	0	10m
wordpress-6698dd7d66-8p55z	1/1	Running	0	27s
wordpress-6698dd7d66-q927g	1/1	Running	0	27s

```
yuvraj_challa@LUCIF3R:~/Lab12$ kubectl delete pod wordpress-6698dd7d66-2vjp8
pod "wordpress-6698dd7d66-2vjp8" deleted from default namespace
yuvraj_singh@LUCIF3R:~/Lab12$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-5x6k6	1/1	Running	0	13m
wordpress-6698dd7d66-8p55z	1/1	Running	0	2m38s
wordpress-6698dd7d66-mgs96	1/1	Running	0	24s
wordpress-6698dd7d66-q927g	1/1	Running	0	2m38s

A new pod will automatically be created to maintain the desired number of replicas.

This demonstrates **Kubernetes self-healing capability**.

Docker Swarm vs Kubernetes

Feature	Docker Swarm	Kubernetes
Setup	Easy	More complex
Scaling	Basic	Advanced
Ecosystem	Small	Large
Industry Use	Rare	Standard

Kubernetes is the **industry standard orchestration platform**. :contentReference[oaicite:9]{index=9}